



Управление памятью

Терминология:

ДП — динамическая память

ДРП — динамическое распределение памяти

ВАП — виртуальное адресное пространство (ВП — виртуальная память)

ФАП — физическое адресное пространство (ФП — физическая память)

Задачи:

- Давать столько, сколько требуется на данный момент (конкуренция между структурами данных — ДРП, конкуренция между процессами - ВП)
- Отсутствие привязки к фиксированным адресам памяти — свободное перемещение по памяти **при сохранении компонент адреса** — сегментация — по отношению к ВАП, ВАП — по отношению к ФАП



Управление памятью

Управление памятью - **распределение адресного пространства как ресурса**

Способы распределения:

- Статическое распределение — фиксированная размерность, привязка при компиляции (к фиксированному адресу)
- Динамическое распределение — во время исполнения программы (runtime)

Виды адресного пространства:

- ВАП процесса — динамическое распределение памяти (**ДРП**)
- ВАП ядра - динамическое распределение памяти
- ФАП — виртуальная память

Потребители ресурса:

- Структуры данных процесса
- Процессы



Динамическое распределение памяти

Виды ДРП:

- ВАП процесса — динамическое распределение памяти (**ДРП**)
- ВАП ядра - динамическое распределение памяти
- Динамическое расширение сегмента ВАП

Ключевые идеи ДРП:

- Исходный (расширяемый) сегмент памяти — «куча» (heap)
- Структура данных — хранение массивов переменной размерности (Си — работа с памятью на низком уровне)
- Проблема — утечка памяти
- Проблема — фрагментация
- Проблема — защита от некорректного использования, конфликт «быстродействие - защита»



Динамическое распределение памяти



```
//-----92-02.cpp
// Система динамического распределения памяти
char *pa; // Область распределяемой памяти – «кучи»
int sz0; // Исходная размерность кучи
void create(int sz){ // Начальное состояние – один свободный блок
    pa=new char[sz];
    *(int*)pa=sz*sizeof(int); // Размерность сво
    sz0=sz; }
```

```
//-----
void _show(){ // Просмотр состоя
char *p=pa;
int lnt;
while(p<pa+sz0){ // Пока не достигли
    lnt=*(int*)p; // Извлечь из-под у
    if (lnt<0){ // Занятый блок - пр
        lnt=-lnt; // Инвертировать дл
        printf("busy:");
    }
    else printf("free:"); // Вывести адрес (
    printf(" addr=%8x sz=%d\n",p,lnt);
    p+=lnt*sizeof(int); // Сдвинуть указате
    }}
```

```
//-----92-02.cpp
// Функция освобождения памяти
void _free(void *q0){
char *q=(char*)q0-sizeof(int);
int lnt=*(int*)q;
lnt=-lnt;
char *pr=NULL,*p=pa;
while(pl=q){
    int ln=*(int*)p;
    if (ln<0) ln=-ln;
    pr=p;
    p+=ln*sizeof(int);
}
*(int*)q=lnt;
if (*(int*)pr>0) {
    *(int*)pr+=lnt*sizeof(int); // Склеить с предыдущим
    pr=pr;
}
lnt=*(int*)p;
q=p+lnt*sizeof(int);
if (q<pa+sz0 && *(int*)q>0)
    *(int*)p+=*(int*)q*sizeof(int);
}
```

Вариант:
последовательность
массивов

// Сместить указатель на счетчик длины
// Извлечь счетчик длины
// pr – указатель на предыдущий блок
// Пока не достигли освобождаемого блока

// Текущий становится предыдущим
// Переход к следующему
// Просмотр до предыдущего
// Освободить блок

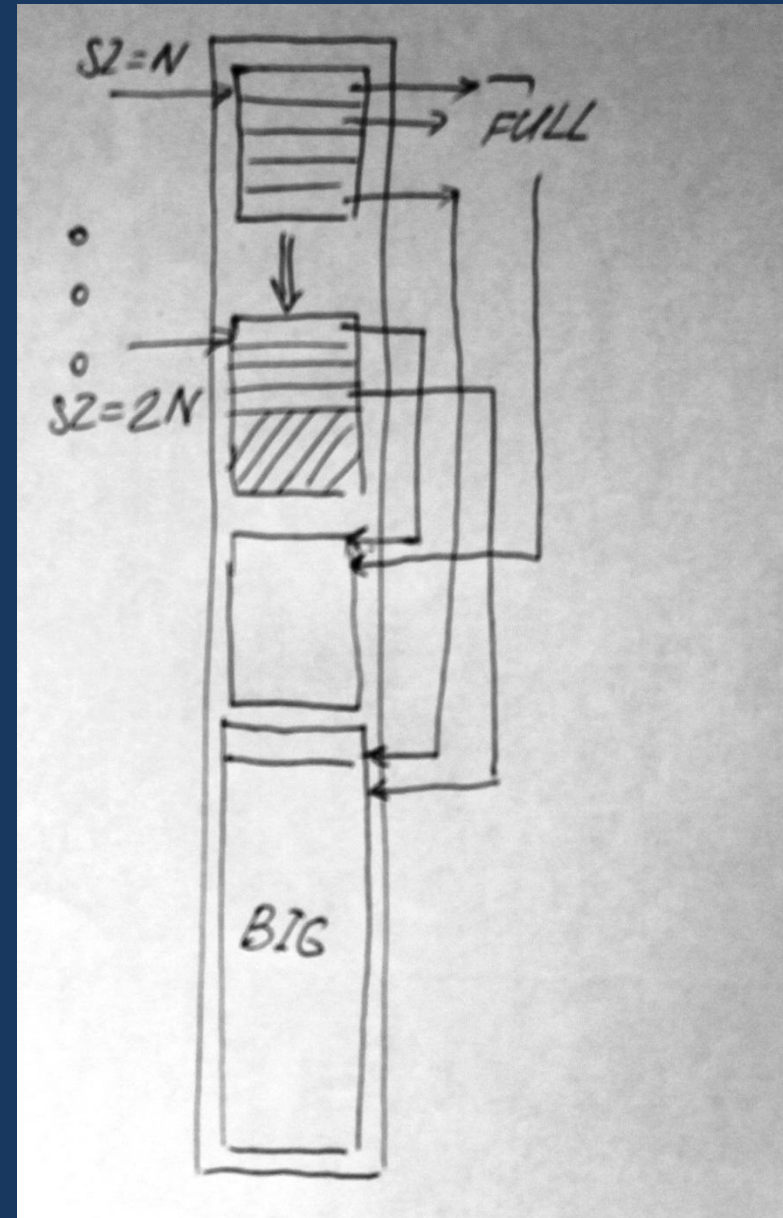
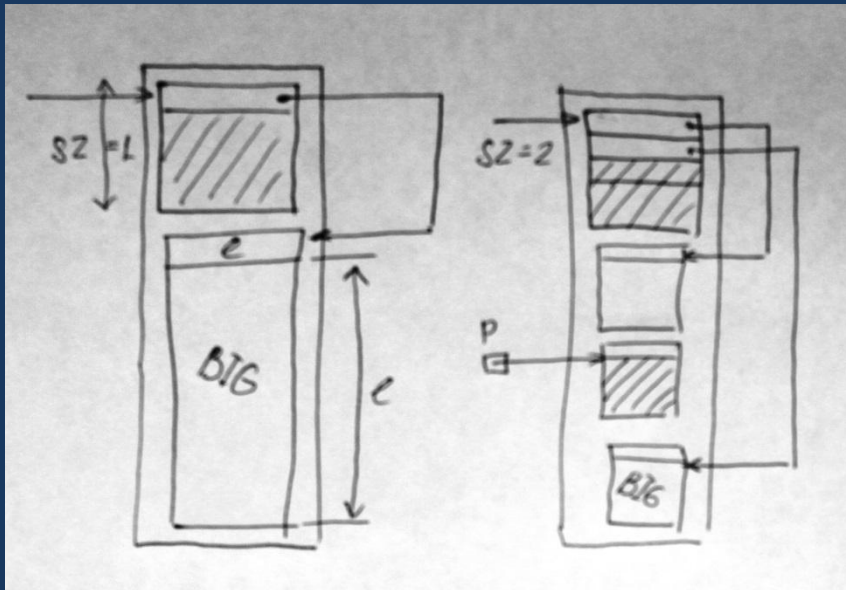
// Есть следующий и он свободен
// Склеить со следующим



Динамическое распределение памяти

Вариант: динамический массив указателей

- ДМУ на блоки, блок — длина + память
- Блок, от которого «отрезаются»
- Поиск: строго подходящий, первый подходящий, наиболее подходящий
- Перераспределение памяти под ДМУ при переполнении (много свободных блоков)
- «Склеивание» соседних свободных

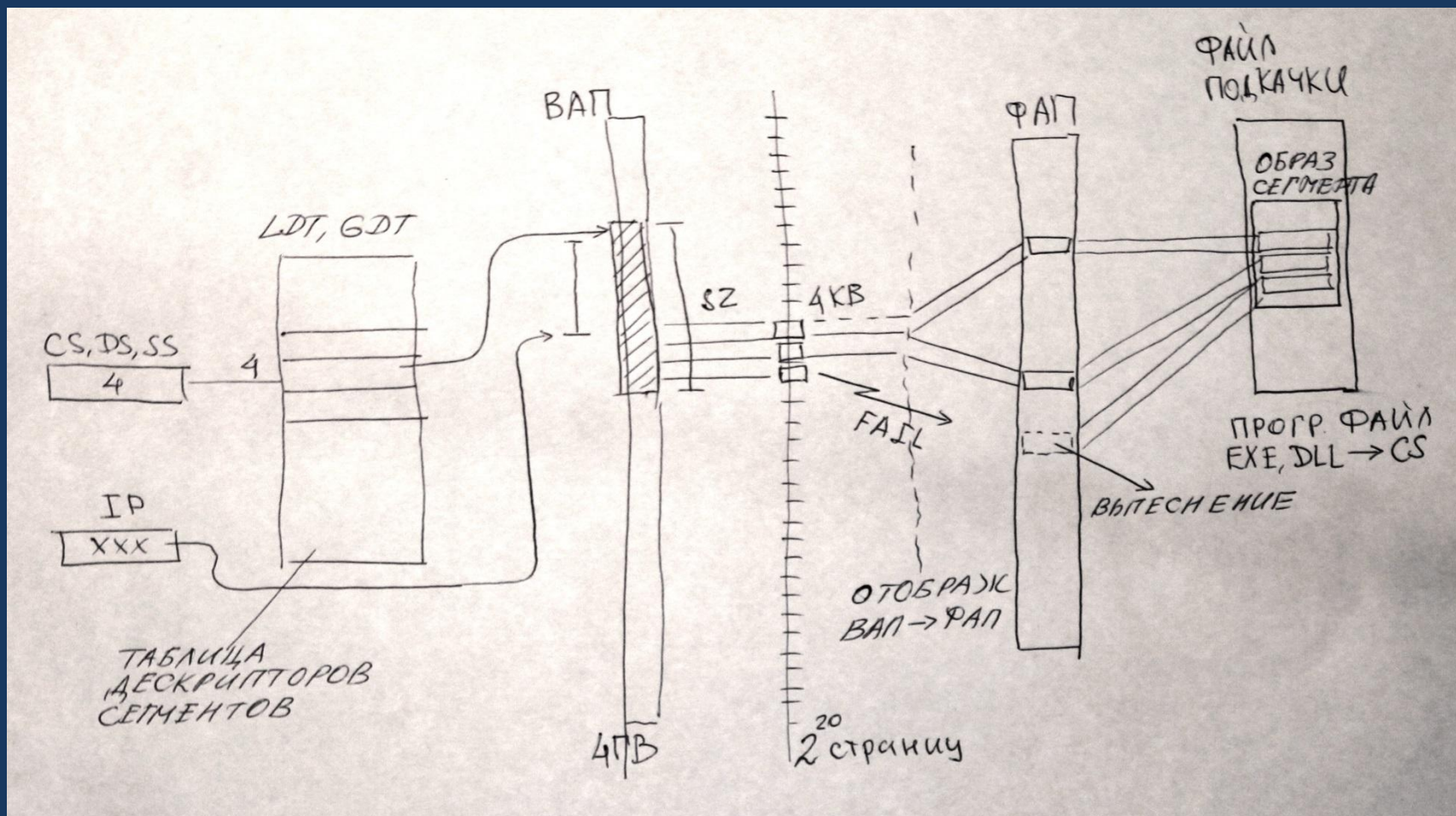




Сегментно-страничное ВАП

Два уровня преобразования адресов:

- Сегментация — относительный адрес внутри сегмента в линейный адрес в ВАП
- Страничное ВАП — отображение блоков (страниц) ВАП на блоки ФАП

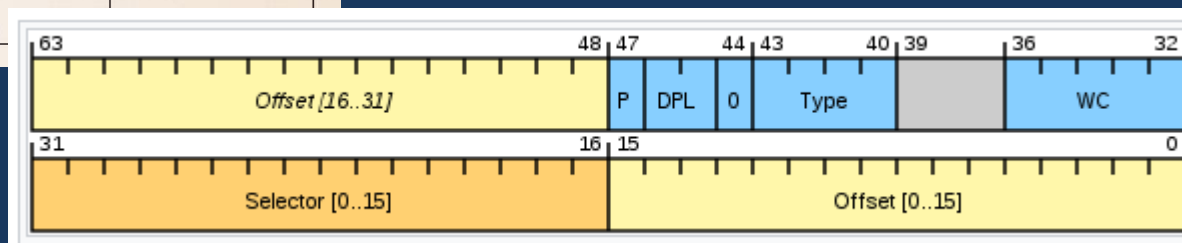
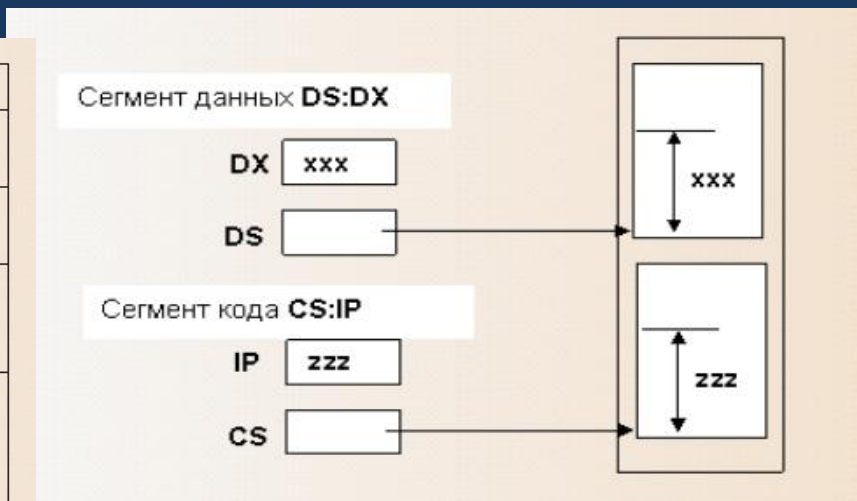




Сегментация

Сегмент - непрерывная область памяти, хранящая данные одного вида (назначения) и имеющая собственную систему относительной адресации и ограничения доступа.

Сегмент	Регистры	Что содержит	Когда создается
Сегмент команд	CS- сегментный, IP- адрес команды	Программный код (операции, операторы)	Трансляция
Сегмент данных	DS – сегментный	Глобальные (статические) данные	Трансляция
Сегмент стека	SS – сегментный, SP- указатель стека	Локальные данные функций, «история» работы программы	При загрузке
Динамическая память	DS – сегментный	Динамические переменные, создаваемые при работе программы	При загрузке, выполнении
Динамически связываемые библиотеки (DLL)	CS- сегментный, IP- адрес команды	Программный код разделяемых библиотек	При загрузке



Сегментные регистры содержат **номер сегмента** — **индекс** в таблице описателей сегментов (глобальной - GDT или локальной - LDT)



Сегментация

Назначение сегментации:

- **Позиционно-независимый код** - перемещение сегмента по АП с перенастройкой базового регистра, относительная адресация сохраняется (перераспределение АП при изменении размеров сегментов)
- **Защита данных на уровне сегмента** (выход смещения за границы, CS — read only)
- Компиляция программных компонент в независимые сегменты — упрощение алгоритмов трансляции (генерации кода)

Сегменты, разделяемые несколькими процессами (DLL)

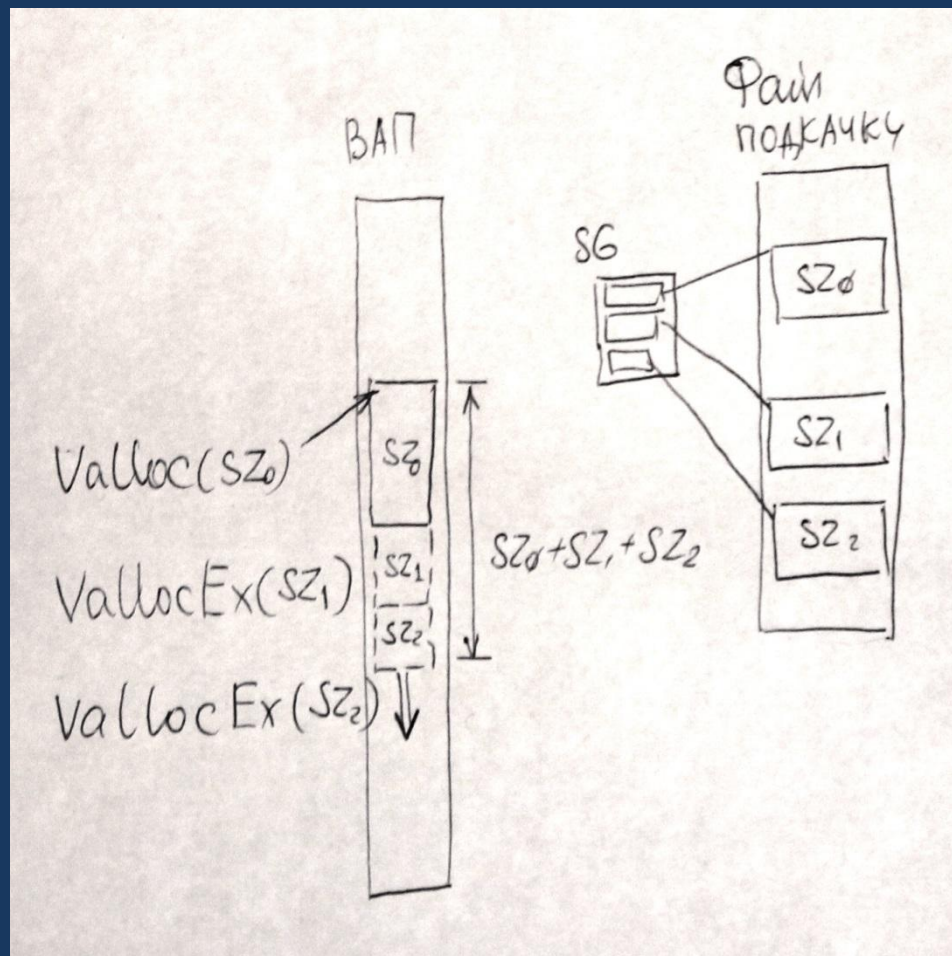
Сегмент — **логическая единица** памяти процесса

«Образ сегмента» - содержимое сегмента в файле (программный файл, DLL, файл подкачки)



Сегменты ВАП - расширение

Динамическое расширение сегмента - резервирование памяти в файле подкачки (образе задачи/процесса)

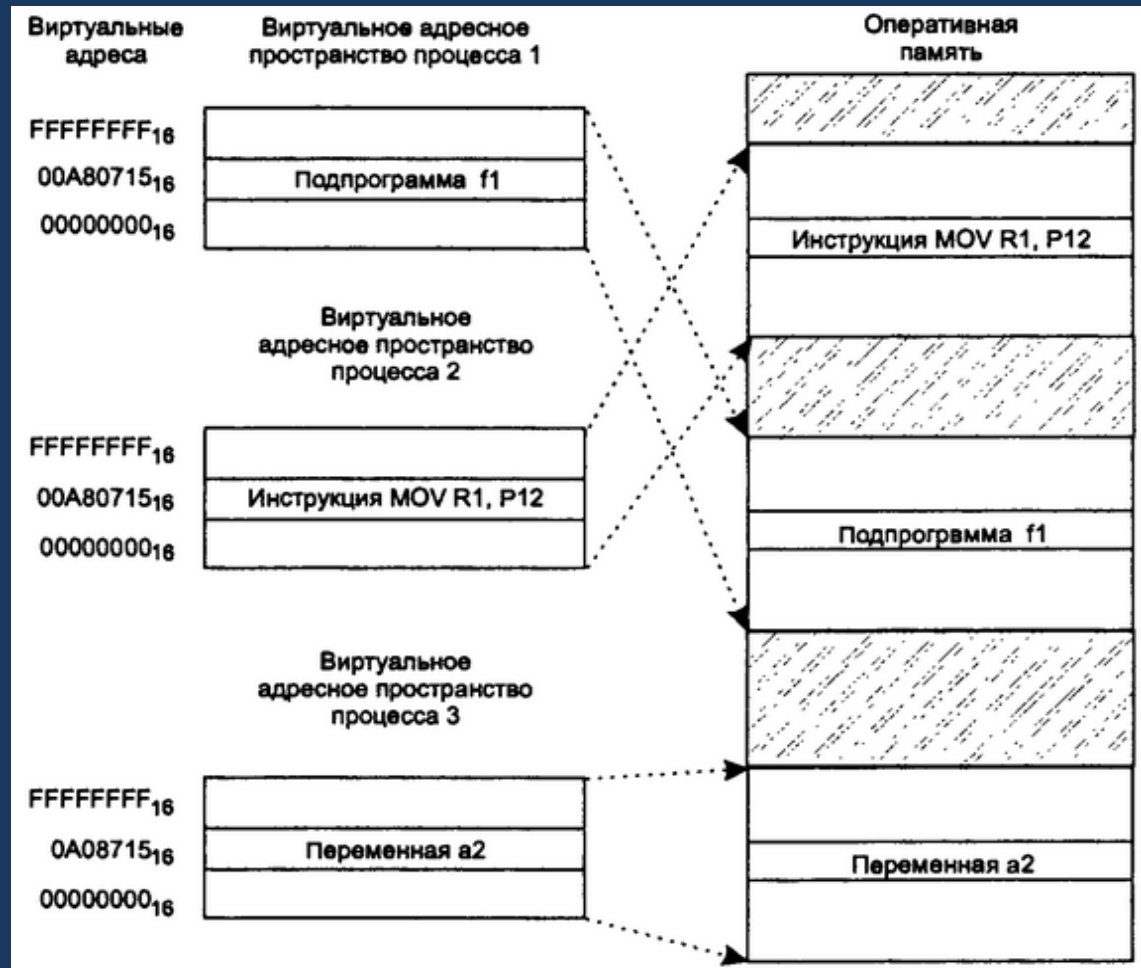




Виртуальная (отображаемая) память

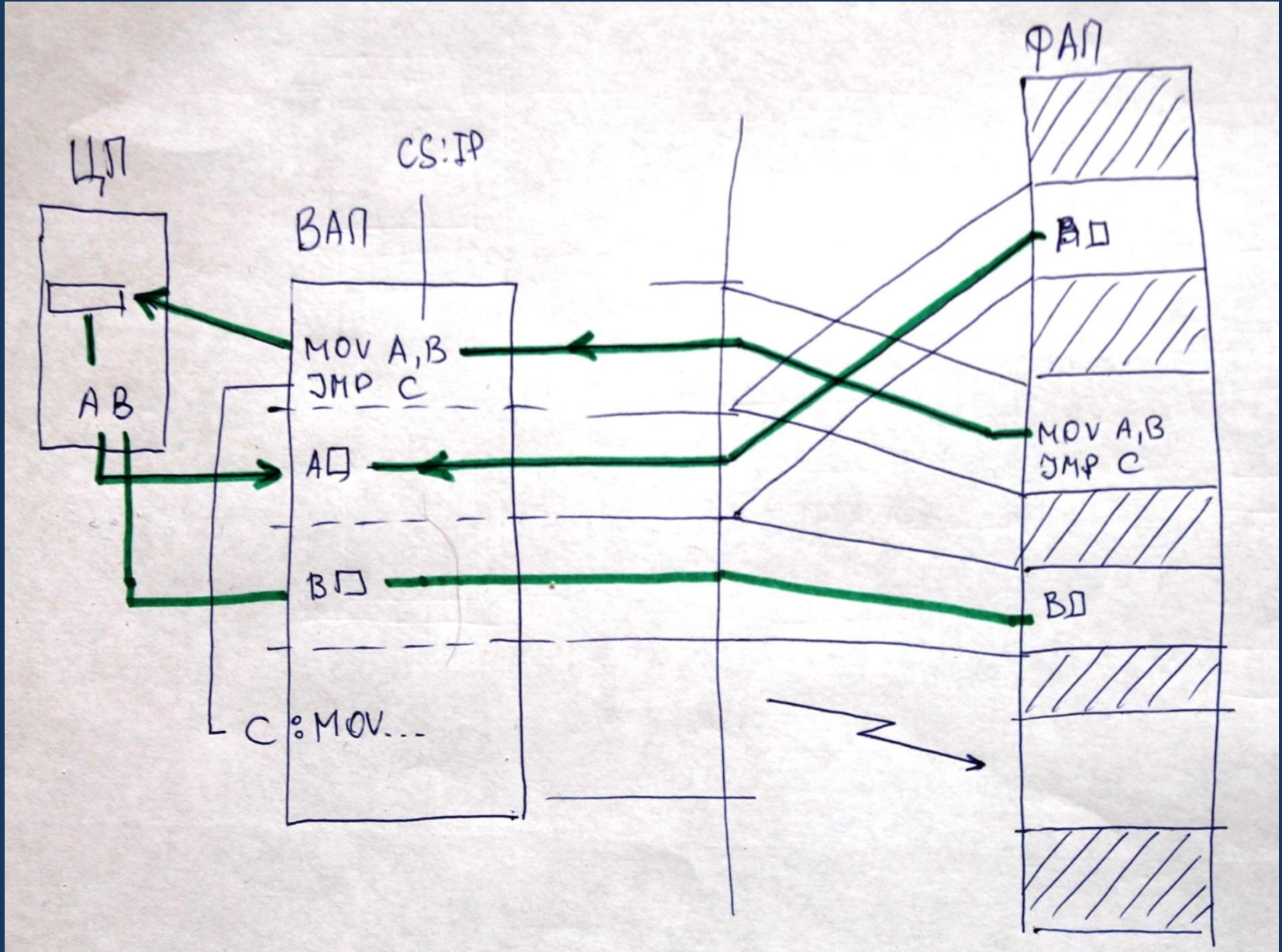
Физическое АП (ФАП) – АП реальных адресов на шине памяти

Виртуальное АП (ВАП) – АП программы (адресов, содержащихся в командах), которое фрагментарно отображается на ФАП блоками по 4Кб (страницами)





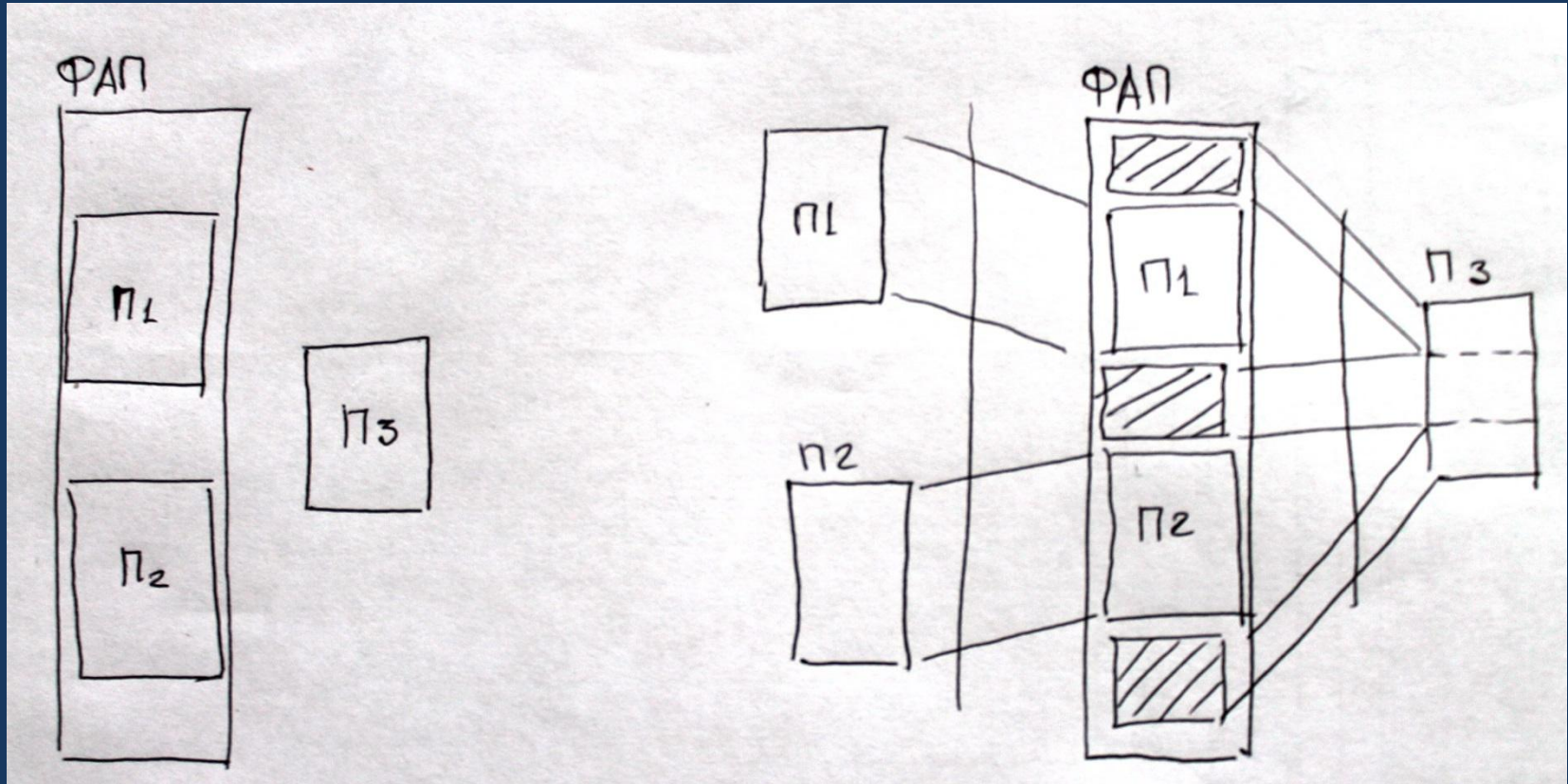
Виртуальная (отображаемая) память





Виртуальная (отображаемая) память

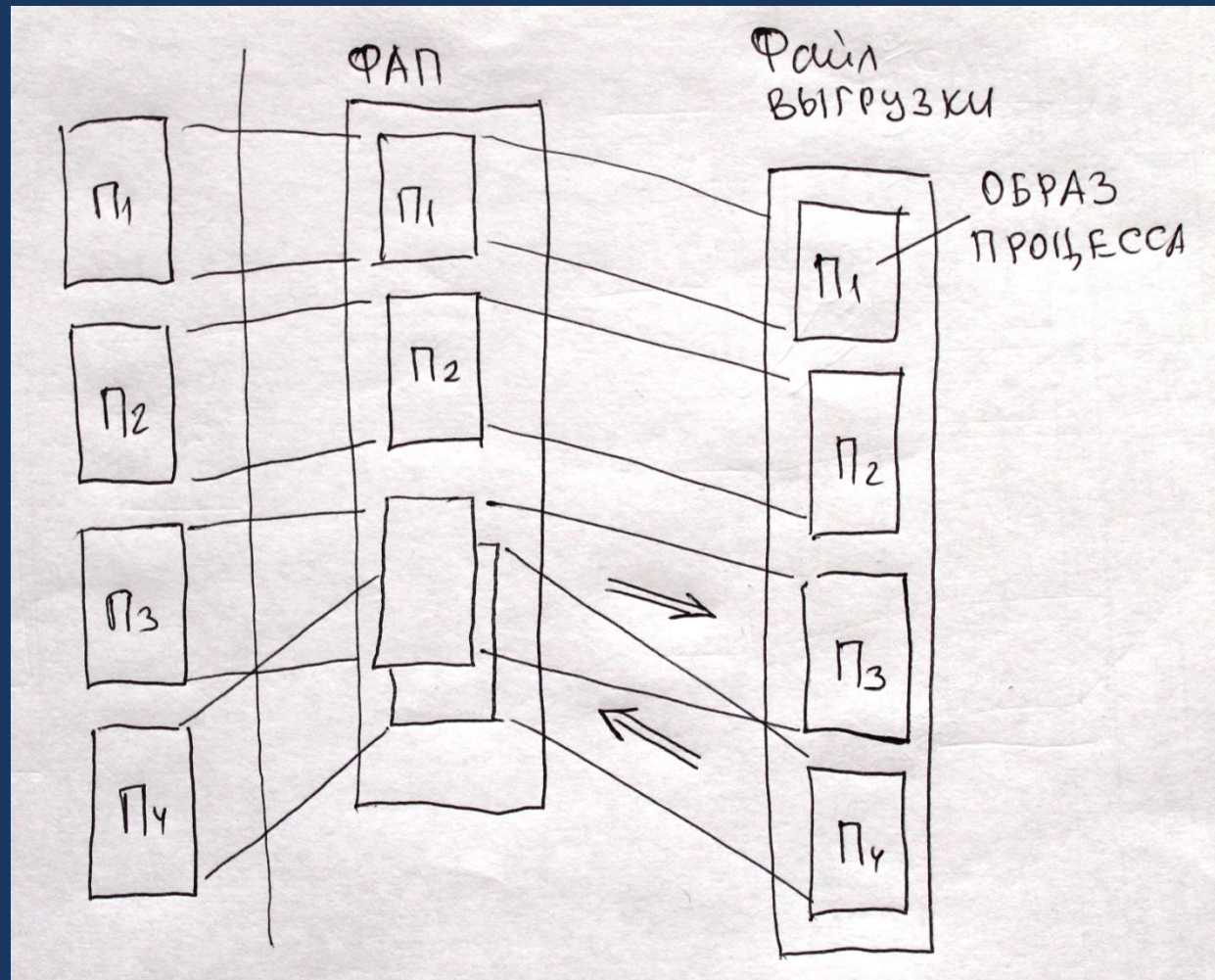
ВП позволяет избежать фрагментацию АП при размещении процессов (задач)





Виртуальная (отображаемая) память

Свопинг (swap — обмен) — обмен «образами задач» между ФАП и файлом выгрузки, аналог разделения времени

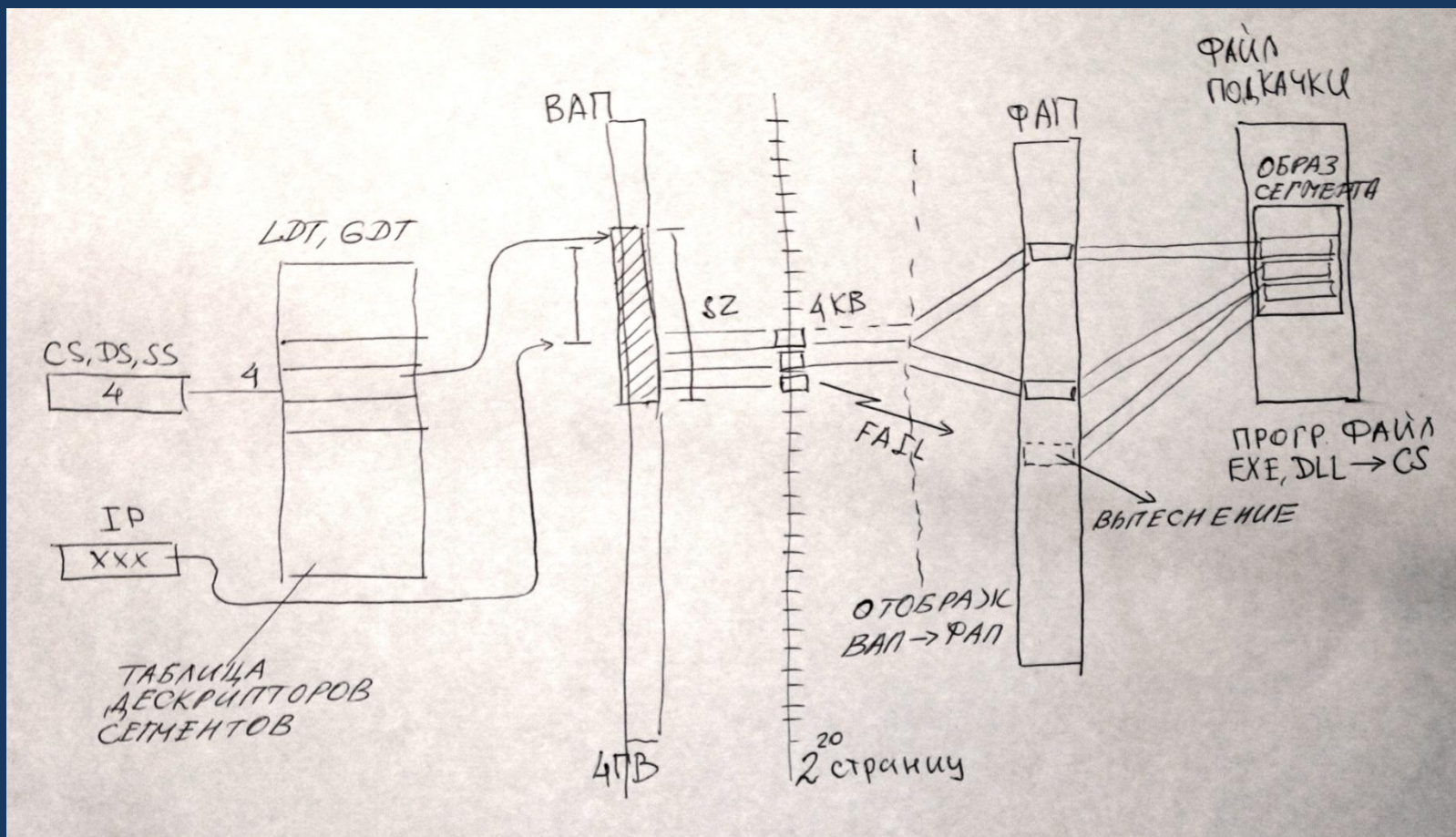




Сегментно-страничное ВАП

Два уровня преобразования адресов:

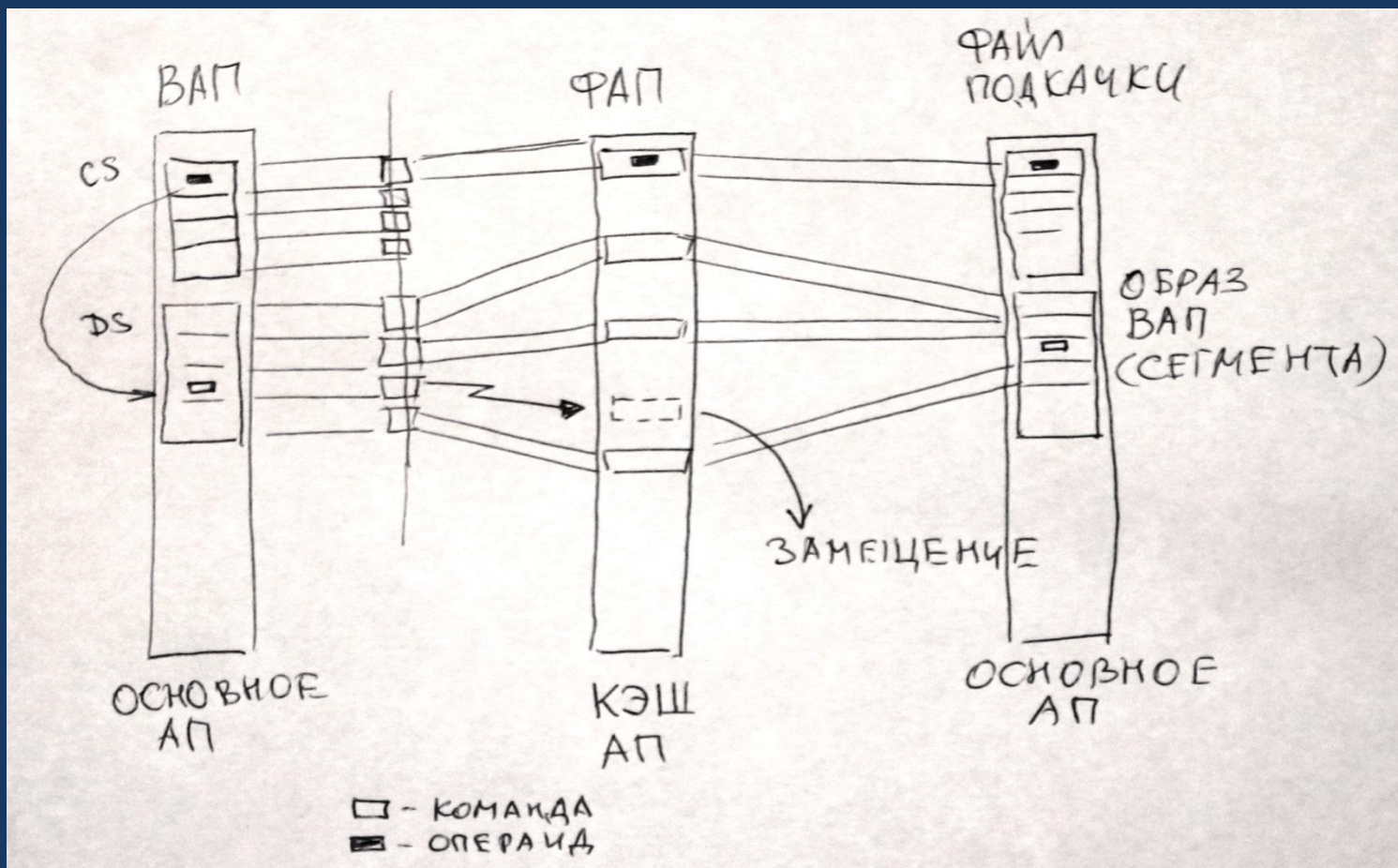
- Сегментация — относительный адрес внутри сегмента в линейный адрес в ВАП
- Страничное ВАП — отображение блоков (страниц) ВАП на блоки ФАП





Виртуальная память

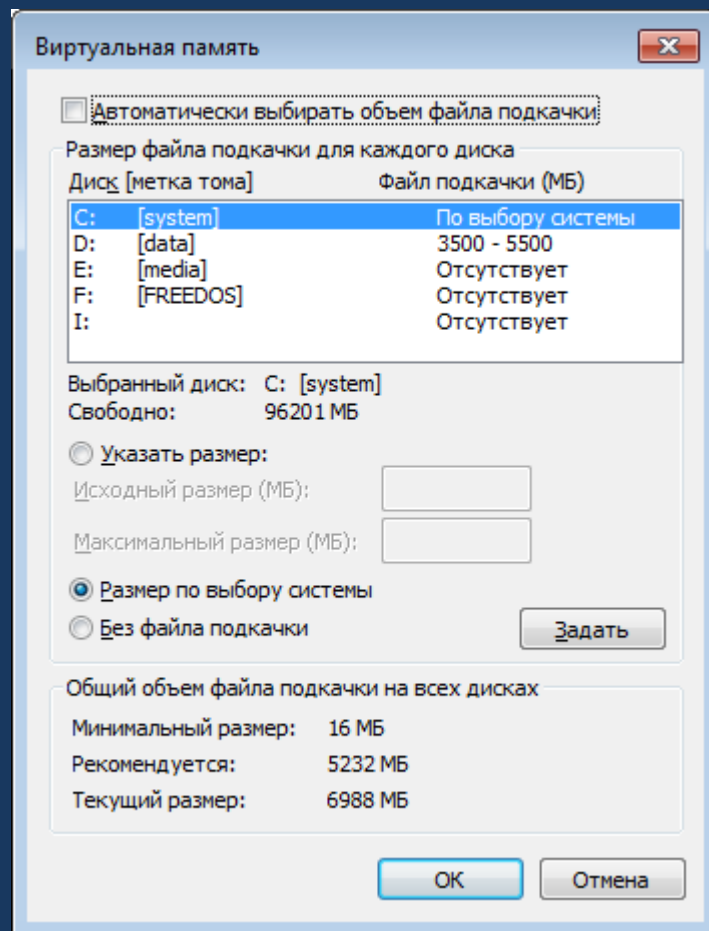
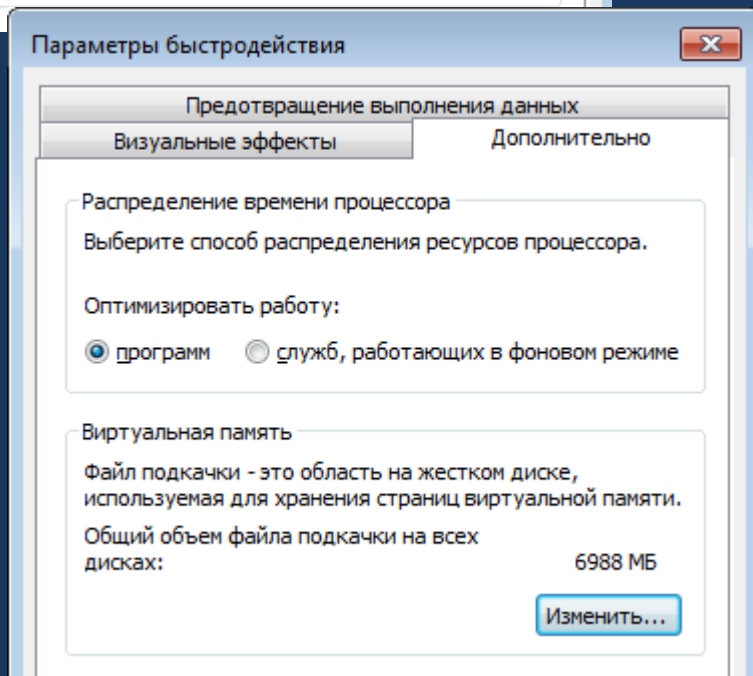
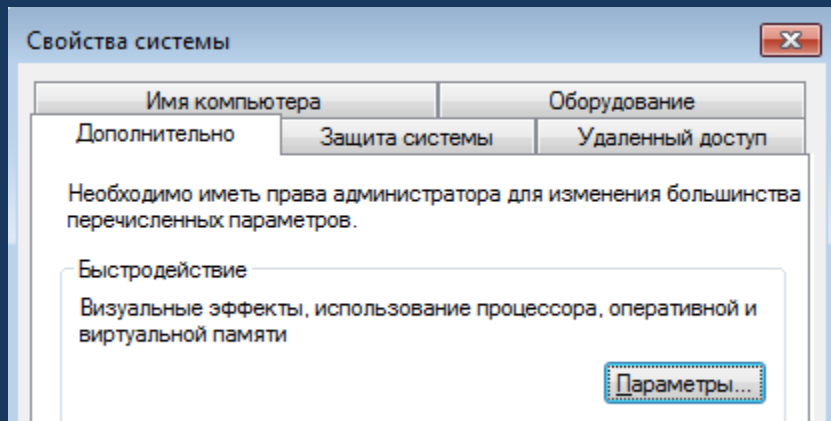
- Разбиение ВАП на блоки (страницы)
- Загрузка страниц «по требованию» - обращение к странице
- Конкуренция страниц за память — между процессами и внутри процесса
- Вытеснение (замещение) страниц, страничное прерывание





Виртуальная память

Страничный файл, файл подкачки, файл выгрузки





Компоненты ВАП ОС

- Собственные данные процесса
- Собственный программный код процесса
- Разделяемый программный код процесса (DLL)
- Резидентная (невытесняемая) часть ядра
 - Структуры данных ядра
 - Системные потоки (управление процессором и памятью)
 - Драйверы
 - HAL — модуль абстрагирования от оборудования (платформенная независимость остального кода)
- Нерезидентная (вытесняемая) часть ядра — ВАП системных процессов в АП ядра
- Системный кэш — резидентный и подкачиваемый



Алгоритм работы ВП

- *Обращение по адресу ВАП из кода программы (либо выборка команды)*
- *Преобразование ВАП в номер страницы и смещение*
- *Выбор из таблицы страниц процесса дескриптора страницы*
- *Установленное отображение = адрес страницы*
 - *Обращение по смещению к странице*
- *Отсутствие отображения — страничное прерывание*
- *Блокирование процесса (потока)*
- *Поиск свободной памяти для загрузки страницы*
- *Вытеснение страницы при отсутствии свободной памяти*
- *Загрузка страницы*
- *Настройка отображения в дескрипторе страницы*
- *Деблокирование процесса*
- *Повторное обращение по адресу ВАП*

Особенности:

- *Аппаратная поддержка + программные компоненты ядра*
- *Система ввода/вывода — страничный обмен*
- *Управление состоянием процесса - блокировка*



Особенности страничного прерывания

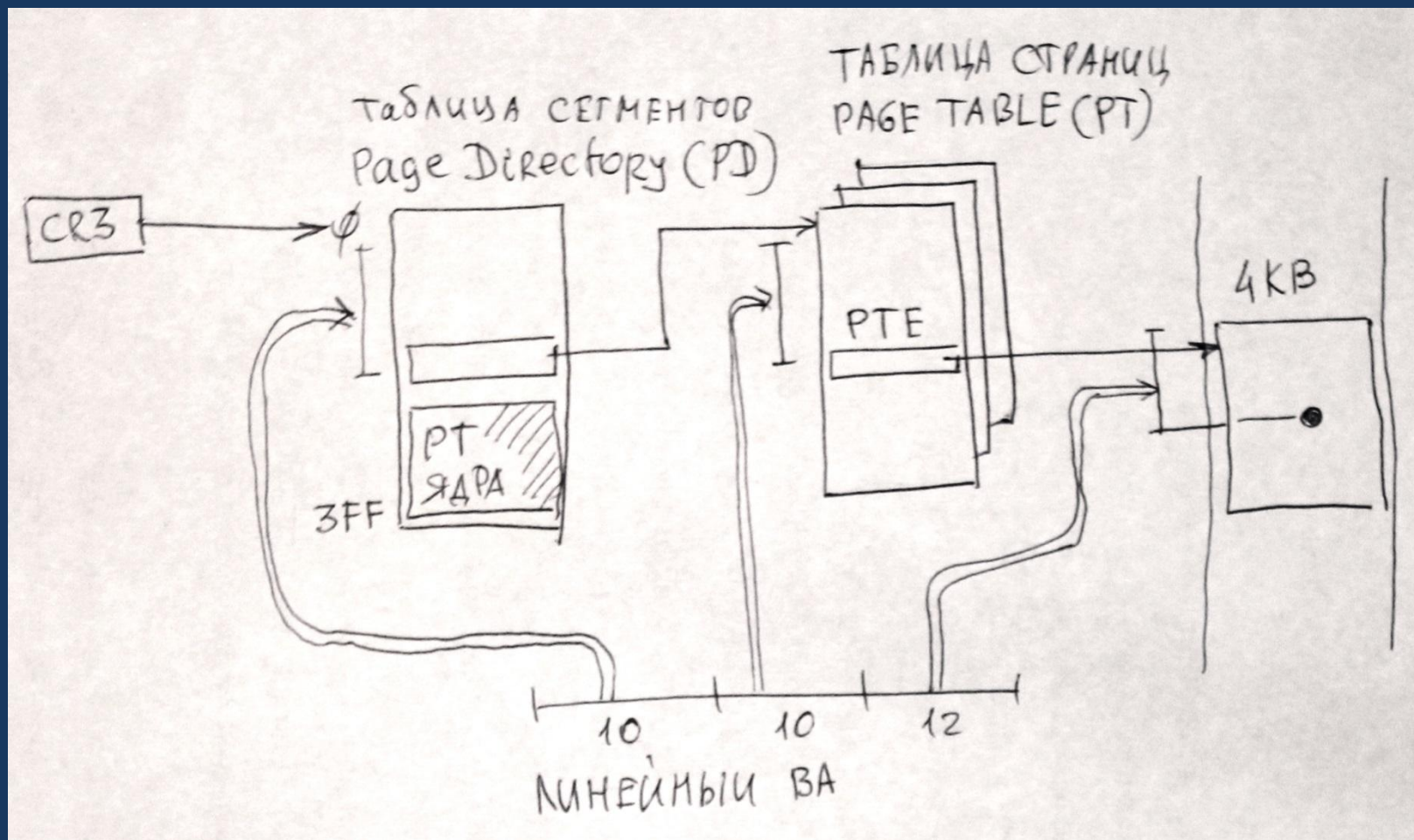
- Происходит внутри цикла выполнения команды (выборка команды, формирование адресов операндов)
- Требуется аппаратный «откат» программно-доступных компонент процессора к состоянию до начала выполнения команды
- *Обычные прерывания обрабатываются процессором между циклами исполнения команд*
- *После выхода из прерывания команду, вызвавшую страничное прерывание, необходимо повторить*



Преобразование виртуальных адресов

Двухуровневый массив дескрипторов страниц в ФП

- CR3 — регистр процессора, адрес каталога страниц (Page Directory)
- PD — 1024 адреса таблиц страниц (Page Table)
- PTE — Page Table Entry — 1024 дескриптора страниц (4B)



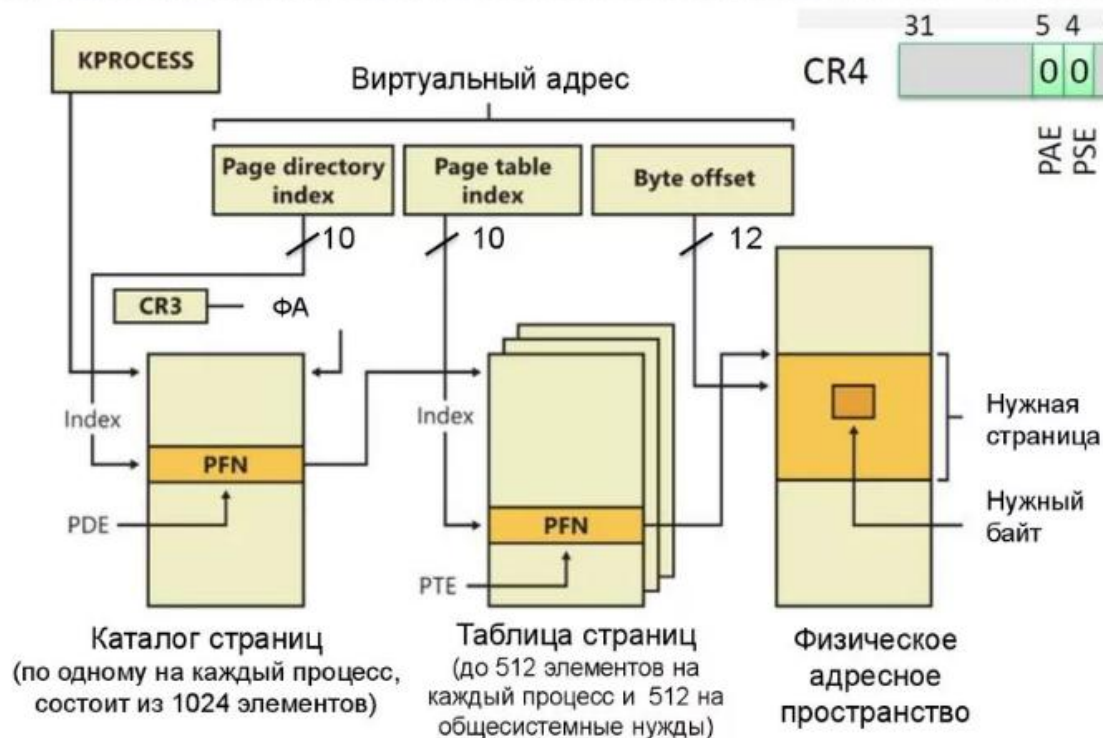


Преобразование виртуальных адресов

Двухуровневый массив дескрипторов страниц в ФП

- CR3 — регистр процессора, адрес каталога страниц (Page Directory)
- PD — 1024 адреса таблиц страниц (Page Table)
- PTE — Page Table Entry — 1024 дескриптора страниц (4B)

Трансляция виртуального адреса в системах x86 (страница 4 КБайт)





Преобразование виртуальных адресов

- Динамическое выделение памяти под массивы фиксированного размера
- Минимально — 1 PD + 1 PT (4Мб ВАП) - 4Кб
- Максимально — 1 PD + 512 PT = $(2^{20}/2)*4B = 2Мб$
- Старшая половина ссылок в PD всех процессов одинакова (отображение ядра)



Преобразование виртуальных адресов

Структура PTE (разряды):

- Page Frame (20) - адрес страницы
- Valid — наличие в ФАП
- Owner — доступна в режиме USER (не ядро)
- Accessed — флаг чтения — было обращение по чтению
- Dirty — флаг изменения — было обращение по записи (изменена)
- Write — разрешение записи
- Cashe Disabled — кэширование запрещено, любая запись вызывает страничное прерывание (сохранение измененных данных)

Варианты обработки страничных прерываний:

- Загрузка страницы
- Ошибка доступа (доступ по записи запрещен) — прерывание - программная ошибка (исключение)
- Немедленное копирование с файл подкачки (Cashe Disabled)



Управление ВП. Стратегии замещения

Тактика: обработка страничного прерывания

Стратегия: замещение страниц, правила конкуренции страниц за память:

- Локальная - страницы процесса конкурируют между собой
- Глобальная - «все против всех»

Алгоритм замещения: определить страницу, удаляемую из памяти

Данные: анализ поведения программы по отношению к памяти —
распределение обращений по адресному пространству

- Свойства алгоритма (анализ)
- Статистика работы программы с памятью



Общее для любой кэш-памяти

КЭШ — прозрачная быстродействующая память ограниченного объема

- Прозрачность — обращение по адресу (имени, идентификатору) к основному пространству (памяти)
- Ограниченность объема — замещение (вытеснение) старых или редко используемых данных
- Чтение из кэша, либо из основной памяти с замещением в кэше

Примеры:

- Кэширование страниц браузером
- Кэширование данных прокси-сервером
- Кэш-память процессора
- ФАП - кэш ВАП = образ процесса в файле подкачки



Алгоритмы замещения

Расписание: известен порядок обращения к страницам. Составляется расписание — порядок вытеснения страниц (перебор вариантов, экспоненциальная трудоемкость)

Оптимальный (OPT): - известен порядок обращения к страницам. Удаляется страница с максимальным временем следующего обращения **в будущем** (линейная трудоемкость). Используется при моделировании для сравнения эффективности алгоритмов вытеснения

LRU (Last Recently Used) — максимально неиспользуемый в прошлом. Интервал с последним обращением к странице. Метафора: платье, которое дольше всего не надевали. «Инверсный» OPT — предполагает проекцию прошлого поведения программы в будущее. Данные: последовательная нумерация обращений к страницам (страница хранит значение счетчика в момент обращения)

FIFO (First In First Out) — обычная очередь, вытесняется наиболее долго находящееся в памяти. Метафора: самое старое платье

RND — случайный, **LIFO** — аномальный, последняя загруженная



Реализация LRU. Часы

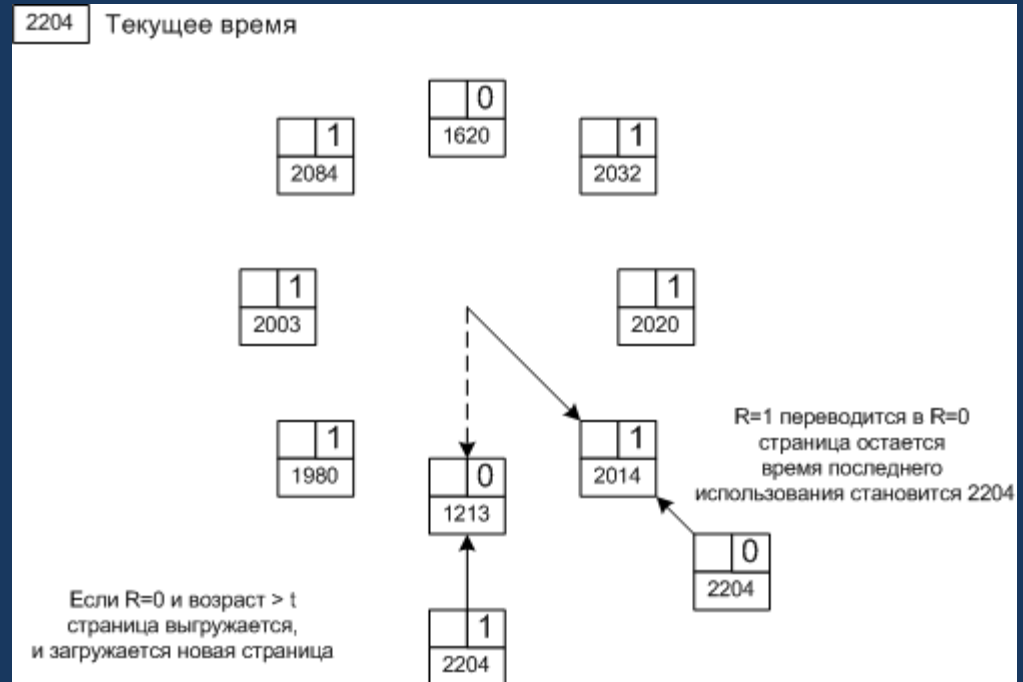
- Используется разряд Accessed в PTE
- Циклический список загруженных страниц
- «Стрелка» бежит по списку и обнуляет разряд Accessed.
- *При обращении к странице Accessed устанавливается аппаратно*
- Вытесняется страница со сброшенным Accessed, ближайшем по ходу стрелки (т. е. неиспользуемом в течение ее оборота)



Реализация LRU. Часы

- Используется разряд Accessed в PTE
- Циклический список загруженных страниц со значением времени, записанного алгоритмом
- «Стрелка» бежит по списку и обнуляет разряд Accessed. Если он был в 1 — устанавливается текущее время
- *При обращении к странице Accessed устанавливается аппаратно*
- Вытесняется страница со сброшенным Accessed с минимальным значением записанного времени

Алгоритм отмечает время, когда он *обнаружил* изменение





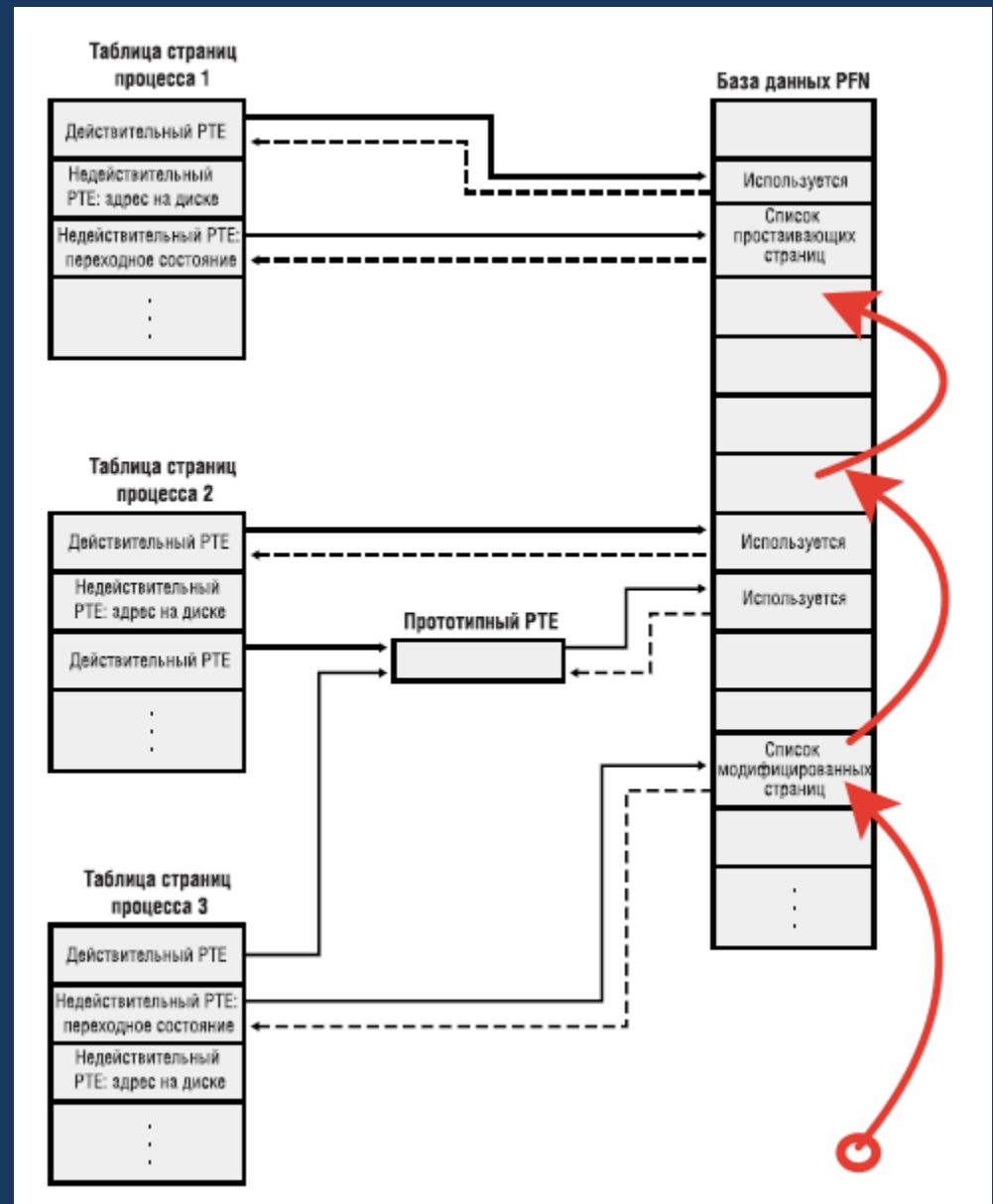
Динамический баланс РН

- РН отслеживается динамически
- Категории страниц: свободные, РН процесса, **переходные** — удалены из РН, но содержат данные процесса
- Переходные участвуют в конкуренции
- Процесс получает дополнительные страницы при обработке страничных прерываний из **свободных и переходных** (содержимое переходных сбрасывается в файл подкачки)
- Ядро периодически «отбирает» у процесса страницы и переводит их в состояние **переходных** (с отключением настройки на них ВАП процесса)
- При повторном обращении к переходной странице и страничном прерывании она возвращается в РН
- Динамическое равновесие в РН между процессами отъема и возвращения страниц



Таблица физических страниц (PFN)

- инверсная таблица описателей страниц (индекс – номер физической страницы)
- описатели страниц с одинаковым состоянием связаны в списки
- цель – поиск физической страницы с нужными параметрами (состоянием)





Условие эффективности ВП

$$\sum V_{pn} < V_{фap}$$

Свопинг РН:

- Блокирование части процессов, чтобы соблюдалось условие эффективности
- Страницы заблокированных процессов замещаются естественным образом
- Через Δt_{swar} смена блокировки процессов
- Δt_{swar} порядка секунд
- Явление пробуксовки $\sum V_{pn} \gg V_{фap}$
 - Выгрузка/загрузка РН — ожидание очереди к диску (файлу подкачки)
 - Чем больше процессов, тем больше время ожидания
 - Время загрузки РН соизмеримо с Δt_{swar} — пробуксовка, скорость исполнения процессов $\rightarrow 0$ при их постоянной перезагрузке
 - Нелинейная деградация производительности



Поведение процессов в памяти

Факторы производительности программ:

- **По времени (процессор). Трудоемкость** — зависимость количества базовых операций алгоритма (обмены, сравнения, рекурсивные вызовы) от размерности СД — N
 - характеристика алгоритма (отчасти программы)
 - архитектурно — независима
 - «грязное время» - аналог трудоемкости для смеси операций
 - оценочный характер трудоемкости — O -нотация (вид функции)
 - функции трудоемкости — const , $\log_2 N$, N , $N \log_2 N$, N^2 , N^m , m^N , $N!$
 - чувствительность к данным: различные виды функций трудоемкости для разных наборов данных (деградация производительности)
- **По памяти:**
 - **избыточность** — требуемый объем памяти в сравнении и исходным объемом СД
 - **распределение обращений по АП** — степень локализации обращений в АП, разница между РН и общим объемом используемого АП



Поведение процессов в памяти

Распределение обращений по АП:

- линейное (сканирование) — последовательные адреса по всему АП (--)
- случайное (псевдо-случайное) по всему АП (-/+)
- локализация в ограниченном объеме АП (++)

Моделирование поведения
алгоритмов сортировки в АП:

- индексы в массиве — адрес ВАП
- ОРТ — предварительная трассировка обращений
- логический анализ характера распределения обращений по ВАП
- эффект приращения ФАП



Моделирование сортировок в виртуальной памяти

Размерность: 1000 простой обмен Сброс

Ячеек на стр.: 5 ОРТ Моделирование

% ФП	чт/зп	прерыв.	% прер.
2	2468820	194810	7,8
5	2406542	184238	7,6
7	2437456	182990	7,5
10	2351800	169442	7,2
15	2430534	166558	6,8
20	2391468	155786	6,5
25	2328324	138600	5,9
30	2379870	133324	5,6
35	2424740	125510	5,1
40	2468314	119388	4,8
45	2454890	108654	4,4
50	2433810	97958	4,0
60	2394610	76640	3,2
65	2346684	65800	2,8
70	2433028	58758	2,4
80	2422178	38720	1,5
85	2410094	29070	1,2
90	2400044	19240	0,8
95	2354158	9400	0,3
97	2439746	5880	0,2
100	2364620	0	0,0

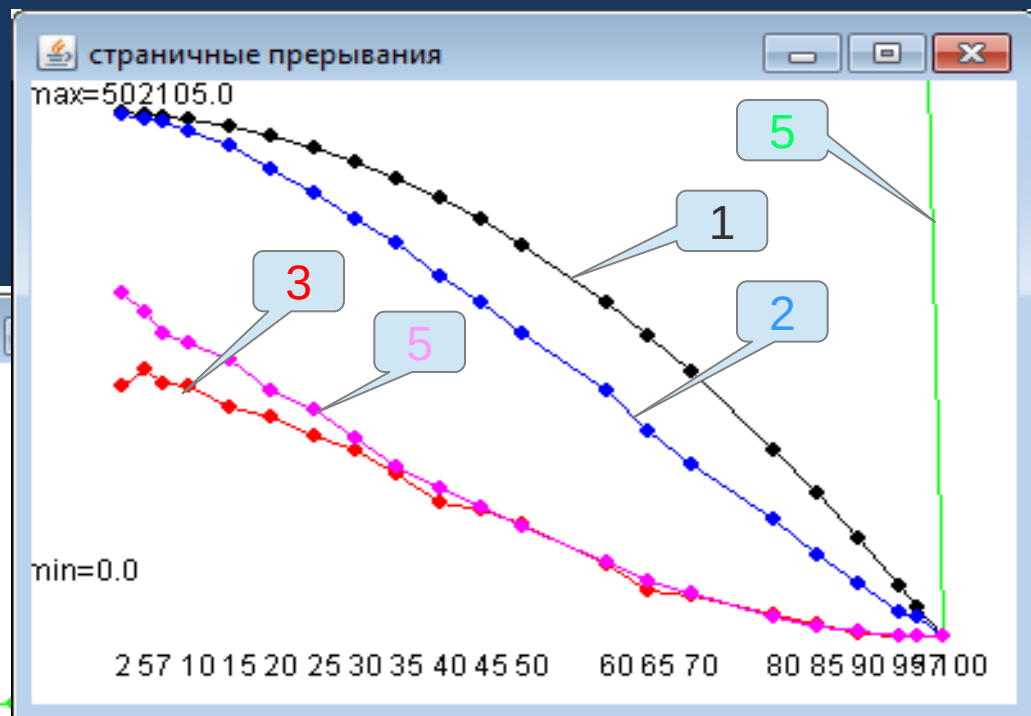
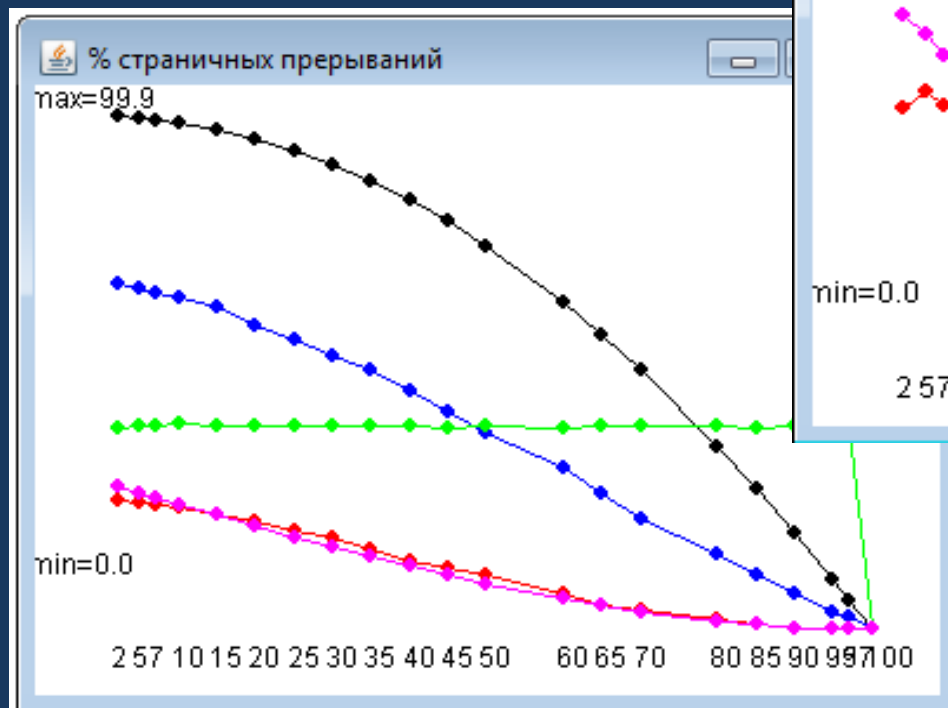
☒ % ФП
☒ чт/зп
☐ запись
☒ прерываний
☒ % прерыв.
☐ % изменений



«Тупые» сортировки

Сканирование по всему или по части АП:

- выбор (1), вставка (2), вставка погружением (3), обмен (4), Шейкер (5)
- Замещение по LRU
- В ВП работают плохо





Рекурсивное разделение

Быстрая сортировка

```
void sort(int in[], int a, int b){
    int i,j,mode;
    if (a>=b) return;
    for (i=a, j=b, mode=1; i < j; mode > 0 ? j-- : i++)
        if (in[i] > in[j]){
            int c = in[i]; in[i] = in[j]; in[j]=c;
            mode = -mode;
        }
    sort(in,a,i-1); sort(in,i+1,b);}
}
```

- медиана – самое левое значение
- количество шагов = длина диапазона -1
- разделение обменом на две части, больших и меньших медианы
- медиана – между частями
- рекурсивный вызов для частей

a				b	mode	до
7	4	9	2	6	1	обмен
6	4	9	2	7	-1	i++
6	4	9	2	7	-1	i++
2	4	9	6	7	-1	обмен
2	4	7	6	9	1	j--
2	4	7	6	9	1	обмен
2	4	6	7	9	-1	i++
a,b						
2	4	6	7	9	Рекурсия	



Рекурсивное разделение

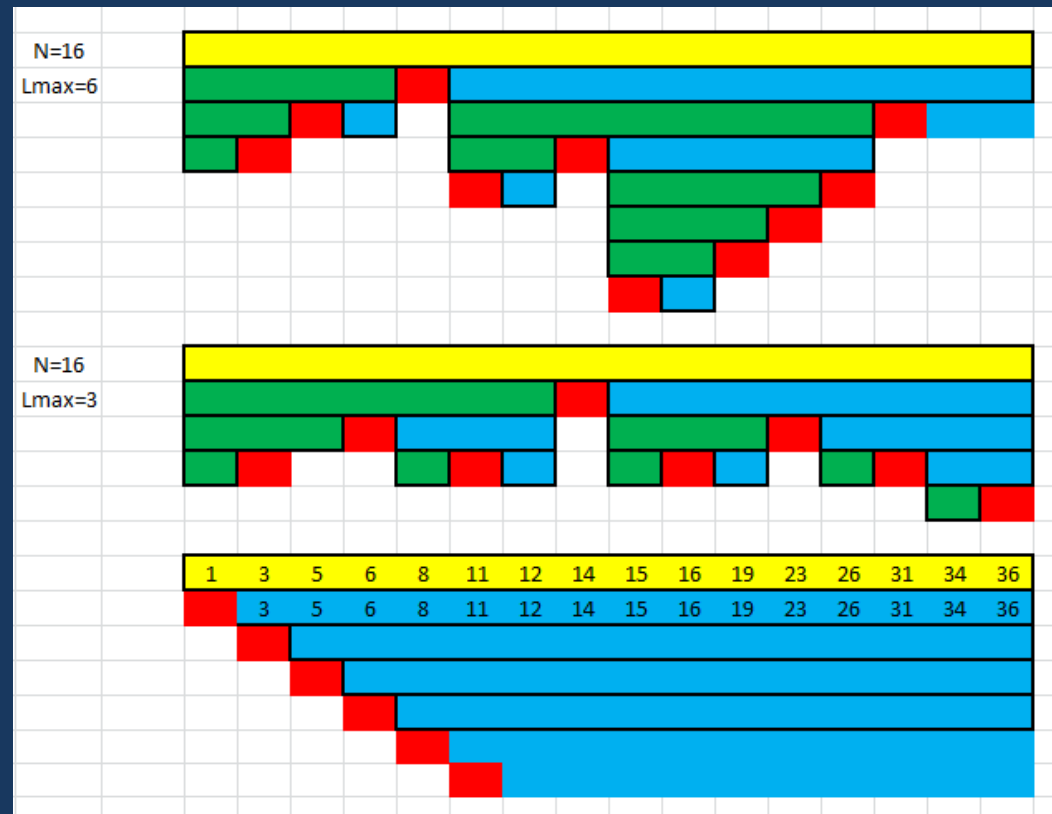
Трудоемкость

$T_{\text{step,min}} = N \log_2(N)$ – идеальное разделение на равные части (без учета медианы), сумма шагов на уровне = N , уровней разделения $\log_2(N)$

$T_{\text{step,mid}} = \dots$

$T_{\text{step,max}} = N^2/2$ – вырожденный случай, медиана слева, сортировка упорядоченного массива

$T_{\text{swap,min}} = 0$ – обменов нет





Рекурсивное разделение

Быстрая — рекурсивное разделение, линейный проход интервала от концов к середине, **локализация** - деление интервала на 2 части

- эффективное приращение ФАП для всех стратегий замещения
- хорошо работает в ВП

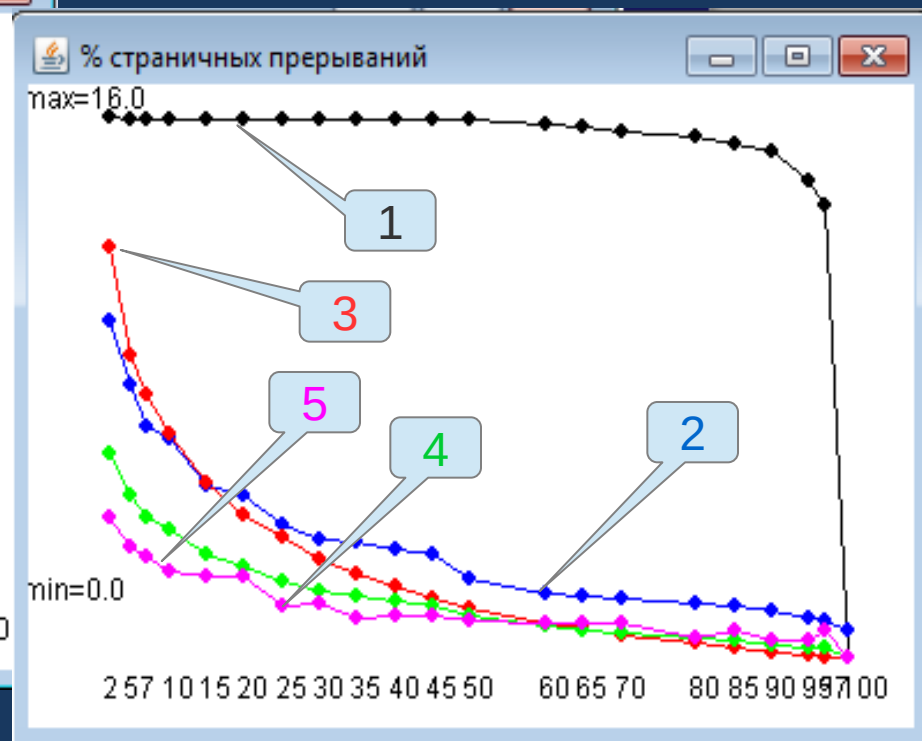
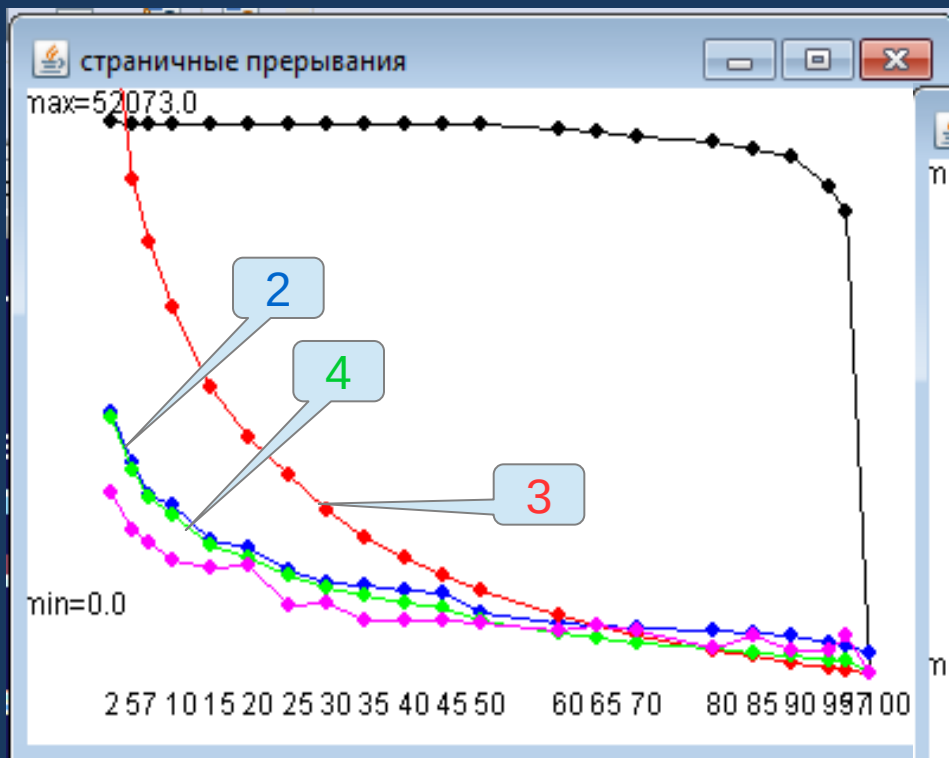




Эффективные сортировки

Эффективные сортировки: трудоемкость $N \log_2 N$

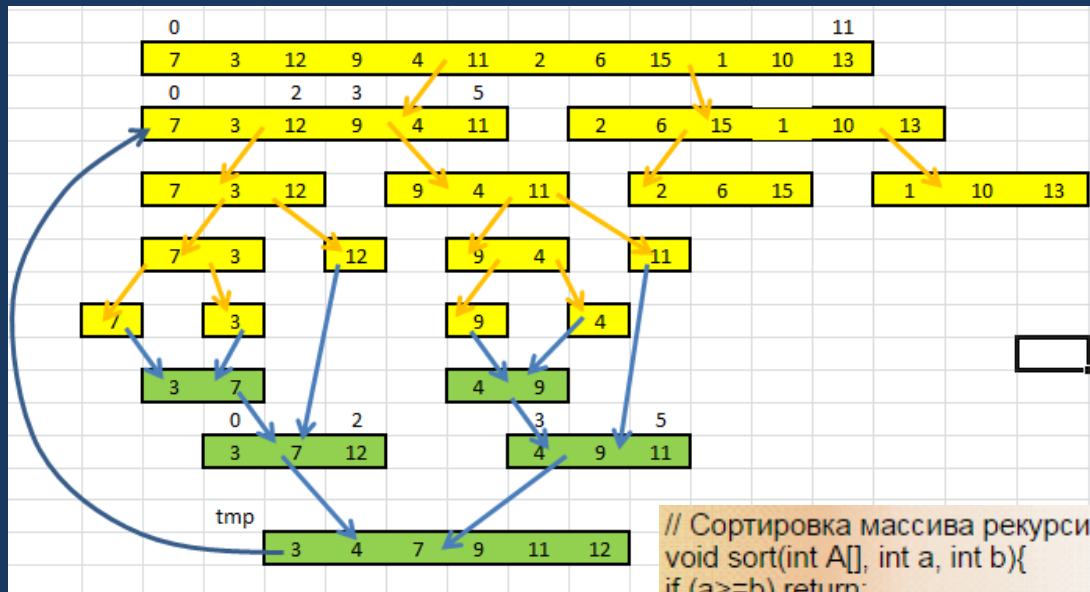
- Циклическое слияние (1), рекурсивное слияние (2), пирамидальная (3), рекурсивное разделение (4), быстрая (5)
- Циклическое слияние — сканирование по всему АП со слиянием группами, пирамидальная — удвоение индекса (локализация по всему АП), рекурсивное слияние — локализация и сканирование групп





Рекурсивное слияние

Рекурсивное слияние – рекурсивное разделение на 2 части, пока не станут ≤ 1 , обратное попарное слияние



$T(N) = O(N \cdot \log_2 N)$ – $\log_2 N$ раз слияние из N элементов, нечувствительна к данным, требуется память для слияния

// Сортировка массива рекурсивным слиянием

```
void sort(int A[], int a, int b){  
    if (a>=b) return;  
    int m=(a+b+1)/2,i,j,k;  
    sort(A,a,m-1);  
    sort(A,m,b);  
    int *tmp=new int[b-a+1];  
    for (i=a,j=m,k=0; k<=b-a; k++)  
        if (i==m || j!=b+1 && A[j]<A[i])  
            tmp[k]=A[j++];  
        else  
            tmp[k]=A[i++];  
    for (i=a,j=0; i<=b; i++,j++)  
        A[i]=tmp[j];  
    delete tmp;  
}
```

// Разделение закончилось

// Нет - взять середину интервала

// Рекурсивный вызов для частей

// Слияние частей во временный массив

// слить из второй части, если

// первая кончилась или во второй меньше

// слить из первой части

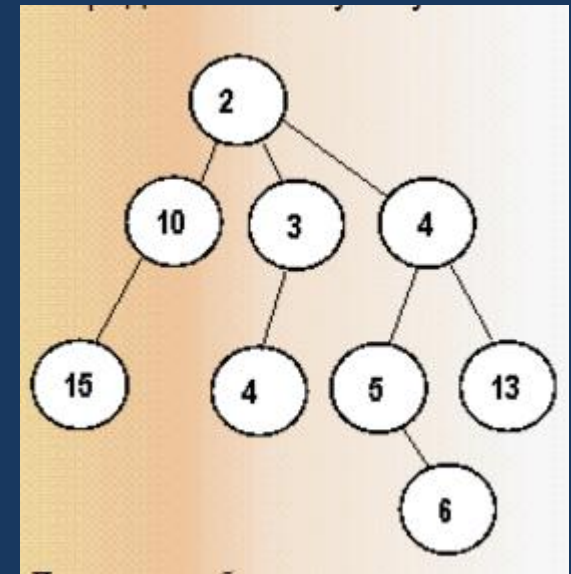
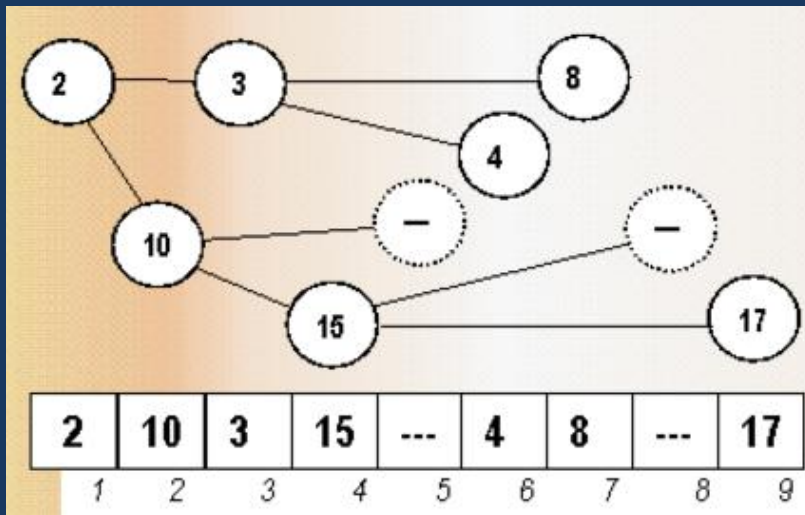
// вернуть слитые части обратно в A

// удалить временный массив



Пирамидальная сортировка

- Дерево, упорядоченное по вертикали — в корневой вершине каждого поддерева — минимальное значение
- Идеи сортировки: включение с вытеснением большего, извлечение с замещением меньшего потомка
- Пирамида - дерево с двумя потомками в массиве $n \rightarrow 2n, 2n+1$ (пирамида)
- Аналогия со вставкой погружением, но шаг **удваивается**
- Трудоемкость — $N \log_2 N$, нечувствительна к данным
- Распределение обращений по АП: **неравномерное, локализовано в начале массива, случайное.**





Рекурсивное разделение

Разделение:

- среднее арифметическое – медиана
- два движка, оставляют слева и справа элементы меньшие и большие медианы
- «упираются» в пару элементов, расположенных **наоборот**
- меняют их местами
- if (i==a) return – для случая «все одинаковые»

```
void sort(int in[], int a, int b){
    int i,j,mode;
    double sr=0;
    if (a>=b) return;
    for (i=a; i<=b; i++) sr+=in[i];
    sr=sr/(b-a+1);
    for (i=a, j=b; i <= j;)
        {
            if (in[i]< sr) { i++; continue; }
            if (in[j]>=sr) { j--; continue; }
            int c = in[i]; in[i] = in[j]; in[j]=c;
            i++,j--;
        }
    if (i==a) return;
    sort(in,a,j); sort(in,i,b);}
"
```